BASE PROJECT PROMPT

Go + React + PostgreSQL + Redis | Full Stack Admin Panel

Dokumen ini berisi prompt lengkap untuk generate base project admin panel fullstack. Paste ke **Claude Code** atau **Cursor Agent** untuk generate seluruh project dari scratch.

OVERVIEW

You are an expert fullstack developer. Generate a complete, production-ready base project with the following specifications. Generate ALL files needed to run the project from scratch without any modification.

1. TECH STACK

Backend

- Language: Go (Golang) 1.21+
- Framework: Gin
- ORM: GORM — dialector pattern so switching DB only requires changing .env DB_DRIVER
- Database: PostgreSQL 15 (default), switchable to MySQL via .env
- Session Store: Redis (go-redis/v9 + gorilla/sessions)
- Session: httpOnly cookie, Secure=true in production, SameSite=Strict
- Auth: Server-side session stored in Redis with TTL
- API Docs: Swagger (swaggo/swag)

Frontend

- Framework: React 18 + Vite
- Router: React Router v6
- HTTP: Axios with withCredentials: true
- State: Zustand (auth, settings, sidebar)
- Data Fetching: TanStack Query v5
- Forms: React Hook Form + Zod validation
- Styling: Tailwind CSS
- Icons: Lucide React
- Notifications: React Hot Toast
- Date: date-fns

2. PROJECT STRUCTURE

```
/
■■■ backend/
```

```
■   ■■■ main.go
■   ■■■ .env.example
■   ■■■ docs/              # swagger generated
■   ■■■ config/
■   ■■■ database/
■   ■   ■■■ migrations/
■   ■   ■■■ seeders/
■   ■■■ middleware/
■   ■■■ modules/
■   ■   ■■■ auth/
■   ■   ■■■ users/
■   ■   ■■■ menus/
■   ■   ■■■ menu_sections/
■   ■   ■■■ roles/
■   ■   ■■■ permissions/
■   ■   ■■■ settings/
■   ■   ■■■ activity_logs/
■   ■■■ utils/
■   ■■■ storage/
■       ■■■ uploads/
■■■ frontend/
    ■■■ src/
    ■   ■■■ api/
    ■   ■■■ components/
    ■   ■   ■■■ ui/
    ■   ■   ■■■ layout/
    ■   ■■■ pages/
    ■   ■   ■■■ auth/
    ■   ■   ■■■ dashboard/
    ■   ■   ■■■ users/
    ■   ■   ■■■ menus/
    ■   ■   ■■■ menu_sections/
    ■   ■   ■■■ roles/
    ■   ■   ■■■ settings/
    ■   ■   ■■■ activity_logs/
    ■   ■■■ hooks/
    ■   ■■■ store/
    ■   ■■■ utils/
    ■   ■■■ router/
```

3. ENVIRONMENT CONFIG (.env.example)

```
APP_NAME=BaseAdmin
APP_ENV=development
APP_PORT=8500
APP_SECRET=change-this-to-32-char-random-string

DB_DRIVER=postgres        # or: mysql
DB_HOST=localhost
DB_PORT=5433              # local dev: 5433 | inside docker network: 5432
DB_USER=postgres
DB_PASS=password
DB_NAME=baseadmin

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASS=
REDIS_DB=0

SESSION_SECRET=change-this-session-secret-32chars
SESSION_MAX_AGE=86400     # 1 day in seconds
SESSION_MAX_CONCURRENT=3  # max active sessions per user
```

```
STORAGE_DRIVER=local      # interface ready for s3
STORAGE_PATH=./storage/uploads
STORAGE_MAX_SIZE=2097152  # 2MB in bytes

MAIL_HOST=
MAIL_PORT=587
MAIL_USER=
MAIL_PASS=
MAIL_FROM=

LOG_CLEANUP_DAYS=90       # auto-delete activity logs older than X days
RATE_LIMIT_LOGIN=5        # max failed login per IP per minute
RATE_LIMIT_API=100        # max requests per IP per minute
```

4. DATABASE MODELS

■ All tables MUST have both id (auto-increment) and uuid fields. Use UUID for API responses, id for internal relations.

Table: users

- id (bigint, PK, auto-increment)
- uuid (uuid, unique, indexed)
- name (varchar)
- email (varchar, unique)
- password (varchar, bcrypt cost 12)
- avatar (varchar, nullable)
- status (enum: active/inactive/suspended)
- last_login_at (timestamp, nullable)
- created_by, updated_by, deleted_by (bigint FK users.id, nullable)
- created_at, updated_at, deleted_at (soft delete)

Table: sessions

- id (bigint, PK)
- uuid (uuid, unique)
- user_id (FK users.id)
- token (varchar, indexed)
- ip_address (varchar)
- user_agent (text)
- expired_at (timestamp)
- created_at

Table: roles

- id (bigint, PK)
- uuid (uuid, unique)
- name (varchar, unique)
- description (text)
- is_superadmin (bool, default false)
- created_by, updated_by, deleted_by (nullable)
- created_at, updated_at, deleted_at

Table: user_roles

- user_id (FK), role_id (FK) — composite PK

Table: menu_sections

- id (bigint, PK)
- uuid (uuid, unique)
- name (varchar)
- icon (varchar)
- order (int)
- created_by, updated_by, deleted_by
- created_at, updated_at, deleted_at

Table: menus

- id (bigint, PK)
- uuid (uuid, unique)
- menu_section_id (FK)
- name (varchar)
- icon (varchar)
- path (varchar — frontend route)
- order (int)
- is_active (bool)
- created_by, updated_by, deleted_by
- created_at, updated_at, deleted_at

Table: permissions

- id (bigint, PK)
- uuid (uuid, unique)
- role_id (FK)
- menu_id (FK)
- can_view, can_create, can_edit, can_delete (bool)

Table: settings

- id (bigint, PK)
- uuid (uuid, unique)
- key (varchar, unique)
- value (text)
- type (enum: string/boolean/file)
- updated_at
- Keys: app_name, app_logo, app_favicon, maintenance_mode

Table: activity_logs

- id (bigint, PK)
- uuid (uuid, unique)
- user_id (FK, nullable)
- action (enum: create/update/delete/login/logout/view)
- module (varchar)
- record_id (varchar, nullable)
- old_value (jsonb, nullable)

- new_value (jsonb, nullable)
- ip_address (varchar)
- user_agent (text)
- created_at

Table: password_reset_tokens

- id (bigint, PK)
- email (varchar, indexed)
- token (varchar, unique)
- expired_at (timestamp)
- used_at (timestamp, nullable)
- created_at

Table: password_histories

- id (bigint, PK)
- user_id (FK)
- password (varchar, bcrypt)
- created_at
- Note: store last 5 passwords, reject reuse

5. API ENDPOINTS (prefix: /api/v1)

All responses use standard format. All list endpoints support: **page, limit, search, sort_by, sort_dir** query params.

Standard Response Format

```
{
  "success": true,
  "message": "Success",
  "data": {},
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "total_pages": 10
  }
}
```

Auth

- `POST /auth/login`
- `POST /auth/logout`
- `GET /auth/me`
- `POST /auth/forgot-password`
- `POST /auth/reset-password`
- `PUT /auth/change-password`

Users

- `GET /users (paginated, search, filter: status, role)`
- `GET /users/:uuid`
- `POST /users`

- PUT /users/:uuid
- DELETE /users/:uuid (soft delete)
- PUT /users/:uuid/status
- POST /users/:uuid/avatar
- GET /users/:uuid/sessions
- DELETE /users/:uuid/sessions/:session_uuid

Menus

- GET /menus (tree with sections, filter: section, is_active)
- GET /menus/:uuid
- POST /menus
- PUT /menus/:uuid
- DELETE /menus/:uuid
- PUT /menus/reorder (bulk order update)

Menu Sections

- GET /menu-sections
- GET /menu-sections/:uuid
- POST /menu-sections
- PUT /menu-sections/:uuid
- DELETE /menu-sections/:uuid
- PUT /menu-sections/reorder

Roles

- GET /roles (paginated)
- GET /roles/:uuid (with permissions)
- POST /roles
- PUT /roles/:uuid
- DELETE /roles/:uuid
- GET /roles/:uuid/permissions
- PUT /roles/:uuid/permissions (bulk update)

Settings

- GET /settings
- PUT /settings (bulk update)
- POST /settings/upload (logo/favicon)

Activity Logs

- GET /activity-logs (filter: user, module, action, date_from, date_to)
- GET /activity-logs/:uuid
- DELETE /activity-logs/cleanup (delete logs older than LOG_CLEANUP_DAYS)

6. MIDDLEWARE STACK

CORS

Whitelist frontend domain only from .env

Helmet

X-Frame-Options: DENY, X-Content-Type-Options: nosniff, HSTS, X-XSS-Protection

Request Size Limit

Max body 10MB

Rate Limiter (Redis)

Login: 5 fails/IP/minute → lockout 15min. API global: 100 req/IP/minute. Forgot password: 3 req/email/hour. Upload: 10 req/user/minute

CSRF

Double submit cookie pattern. Skip for GET/HEAD/OPTIONS

Session Auth

Validate session from httpOnly cookie via Redis. Attach user to context

Maintenance Mode

Block all requests if maintenance_mode=true. Cache setting in Redis 60s. Bypass superadmin

Permission Guard

Each route tagged with menu path + action. Check role permissions. Superadmin bypasses all. Return 403 if denied

Activity Logger

Async via goroutine + buffered channel. Non-blocking. Capture IP, user agent, old/new value

XSS Sanitizer

Strip HTML tags from all string inputs before processing

7. ACTIVITY LOG IMPLEMENTATION

Use goroutine + buffered channel for async, non-blocking logging.

```
// Helper function signature
LogActivity(ctx context.Context, action string, module string,
            recordID string, oldValue interface{}, newValue interface{})

// Usage in handlers
oldUser := getUserByID(id)
updateUser(id, payload)
newUser := getUserByID(id)
go LogActivity(ctx, "update", "users", uuid, oldUser, newUser)

// Auto-cleanup: run as cron job at midnight
// Delete records where created_at < NOW() - INTERVAL '90 days'
// Configurable via LOG_CLEANUP_DAYS env
```

Audit Trail

Every create/update/delete operation MUST log:

- created_by: user UUID who created the record
- updated_by: user UUID who last updated

- deleted_by: user UUID who soft-deleted
- All stored in activity_logs with full old/new JSON diff

8. SECURITY SPECIFICATIONS

Session & Auth

- Redis session store with TTL = SESSION_MAX_AGE
- Session rotation on every login (invalidate old sessions)
- Max concurrent sessions per user = SESSION_MAX_CONCURRENT (default 3)
- Session fingerprinting: bind to IP + User-Agent hash
- Cookie: HttpOnly=true, Secure=true (production), SameSite=Strict
- Force logout: delete Redis session key immediately

Password Policy

- Min 8 chars, must contain uppercase, number, symbol
- Bcrypt cost factor: 12
- Password history: reject last 5 passwords (store in password_histories)
- Reset token: expires 1 hour, one-time use only (mark used_at on use)

Rate Limiting (Redis-based)

- Login: 5 failed attempts/IP/minute → lockout 15 minutes
- Forgot password: 3 requests/email/hour
- API global: 100 requests/IP/minute
- Upload: 10 requests/user/minute
- Log all failed login attempts to activity_logs

API Security

- CSRF: double submit cookie pattern
- CORS: whitelist only from FRONTEND_URL env
- Helmet headers on all responses
- Request body size limit: 10MB
- SQL injection: safe via GORM parameterized queries (automatic)
- XSS: sanitize all string inputs before DB insert
- File upload: validate MIME type (not just extension), store outside webroot
- File upload: max 2MB, allowed types: image/jpeg, image/png, image/webp
- Unique filename: UUID + original extension

Infrastructure

- .env in .gitignore, .env.example committed
- APP_SECRET and SESSION_SECRET min 32 chars random
- Superadmin role: hardcoded, cannot be deleted or demoted
- Superadmin bypasses ALL permission checks

9. FRONTEND SPECIFICATIONS

Design System

```
Colors:
  Primary:        #2563EB  (blue-600)
  Primary Dark:   #1D4ED8
  Sidebar BG:     #0F172A  (slate-900)
  Sidebar Text:   #94A3B8  (slate-400)
  Sidebar Active: #2563EB
  Background:     #F8FAFC  (slate-50)
  Card:           #FFFFFF
  Border:         #E2E8F0
  Text Primary:   #0F172A
  Text Secondary: #64748B
  Danger:         #EF4444
  Success:        #22C55E
  Warning:        #F59E0B

Dark Mode: full support via Tailwind dark: prefix + class toggle on <html>
Font: Inter (Google Fonts)
```

Layout

- Sidebar: collapsible on mobile (hamburger), fixed on desktop
- Top navbar: breadcrumb, dark mode toggle, user avatar dropdown
- Sidebar menu: dynamic from API, filtered by user permissions
- Menu sections as sidebar group headers with icons
- Login page: completely separate layout (no sidebar)

Pages

/login

Email/password form, show app logo & name from settings, no sidebar

/forgot-password

Email input, send reset link

/reset-password

New password + confirm, validate token

/

Dashboard: stats cards (users, menus, roles, logs), recent activity table (last 10)

/users

Table: avatar, name, email, status badge, roles, last login. Search, filter, sort, pagination. Add/Edit modal. Delete confirm. View/kick sessions

/menus

Table grouped by section. Drag-to-reorder. Add/Edit modal with icon picker

/menu-sections

Table: name, icon, order, menu count. Add/Edit modal

/roles

Table: name, description, user count, superadmin badge

/roles/:uuid/permissions

Permission matrix: rows=menus, cols=can_view/create/edit/delete. Checkbox grid. Select all per row

/settings

Tabs: General (app name), Appearance (logo/favicon upload with preview), Maintenance (toggle)

/activity-logs

Table: time, user, action badge, module, record ID, IP. Click row → modal with old/new JSON diff side-by-side. Filter by user/module/action/date

/403

Forbidden page

/404

Not found page

Global Frontend Features

- Permission guard: hide buttons if no permission (usePermission hook)
- Route guard: redirect to /403 if accessing page without can_view
- Skeleton loading for ALL tables and forms
- Toast notification (success/error/warning) via React Hot Toast
- Confirm dialog component (reusable) before every delete
- Empty state component with icon + message
- Table: sort columns, filter, pagination (page/limit selector)
- Form validation client-side via Zod schemas
- Dark mode toggle (persist preference in localStorage)
- App name & logo loaded from settings API on init, stored in settingsStore
- Maintenance mode: if enabled, show maintenance page (except superadmin)
- Axios: if 401 → redirect to /login, if 503 → redirect to maintenance

Zustand Stores

```
authStore:     { user, roles, permissions, isAuthenticated, setUser, logout }
settingsStore: { appName, logo, favicon, maintenanceMode, fetchSettings }
sidebarStore:  { isCollapsed, activeMenu, toggle }

usePermission() hook:
  canView('menus')    → checks authStore permissions
  canCreate('users')  → checks authStore permissions
  canEdit('roles')    → checks authStore permissions
  canDelete('users')  → checks authStore permissions
```

10. SWAGGER / API DOCUMENTATION

- Use swaggo/swag library
- Annotate all handlers with @Summary, @Tags, @Accept, @Produce, @Param, @Success, @Failure, @Router
- Serve at /api/v1/docs (swagger UI)
- Generate with: swag init -g main.go
- Include bearer token auth in swagger config

```
// main.go annotation example:
// @title           BaseAdmin API
// @version         1.0
// @description     Admin Panel Base Project API
// @host            localhost:8080
// @BasePath        /api/v1
// @securityDefinitions.apikey SessionCookie
// @in cookie
// @name session
```

11. STORAGE INTERFACE

```
type StorageInterface interface {
    Upload(file multipart.File, header *multipart.FileHeader, folder string) (string, error)
    Delete(path string) error
    GetURL(path string) string
}

// Implement: LocalStorage
// Interface ready for: S3Storage (swap via STORAGE_DRIVER env)

// Validation before upload:
// 1. Check MIME type (not just extension) using http.DetectContentType
// 2. Max size from STORAGE_MAX_SIZE env
// 3. Generate filename: uuid + original extension
// 4. Store outside webroot
```

12. SEEDER

Run with: `go run main.go seed`

- Superadmin role (is_superadmin: true, cannot be deleted)
- Default menu sections: Dashboard, User Management, System
- Default menus matching all frontend pages
- All permissions set to true for superadmin role
- Superadmin user: admin@example.com / Admin@12345
- Default settings: app_name='BaseAdmin', maintenance_mode='false'

Run migration with: `go run main.go migrate`

13. DOCKER COMPOSE

```
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: baseadmin
```

```
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    ports: ["5433:5432"]   # 5433 host → avoid conflict with local PostgreSQL on 5432
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    command: redis-server --maxmemory 128mb --maxmemory-policy allkeys-lru
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]

  backend:
    build: ./backend
    ports: ["8500:8500"]
    environment:
      DB_HOST: postgres
      DB_PORT: 5432        # internal docker network — always 5432
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }
    env_file: ./backend/.env

  frontend:
    build: ./frontend
    ports: ["8501:80"]
    depends_on: [backend]

volumes:
  pgdata:
```

14. IMPORTANT IMPLEMENTATION NOTES

- All tables have BOTH id (bigint auto-increment) AND uuid (uuid). Use UUID in all API responses and URL params. Use id for internal DB relations (FK performance).
- GORM dialector pattern: config/database.go reads DB_DRIVER env and returns correct dialector. Switching DB = only change .env.
- Activity log goroutine must NEVER block main request. Use buffered channel (size 1000).
- Maintenance mode setting: cache in Redis with 60s TTL. Do not query DB on every request.
- Superadmin: bypass all permission checks in middleware. Cannot be deleted, status cannot be changed, role cannot be unassigned.
- Password change: validate against last 5 passwords in password_histories table.
- All soft-deleted records excluded from GET responses automatically via GORM.
- created_by/updated_by/deleted_by: store user ID from session context.
- API versioning: all routes under /api/v1. Structure ready for /api/v2 later.
- CORS: only allow FRONTEND_URL from env. No wildcard in production.
- README.md must include: prerequisites, setup steps, env config, docker one-liner, default credentials, folder structure, swagger URL.

END OF PROMPT — Paste everything above into Claude Code or Cursor Agent mode to generate the full project.

Default credentials after seeding: `admin@example.com` / `Admin@12345`

Swagger UI: `http://localhost:8500/api/v1/docs`

Frontend: `http://localhost:8501`

■ Port notes: Backend=8500, Frontend=8501, PostgreSQL host=5433 (mapped dari 5432 internal docker). Jika run backend di luar docker, gunakan DB_PORT=5433. Jika backend di dalam docker, gunakan DB_PORT=5432 (internal network). PostgreSQL lokal di 5432 tidak akan bentrok.